Chapter 09

Ansible: Configuration Management & Post-Provisioning

Turning 12 blank virtual machines into a fully configured, hardened, service-running lab environment — without a single manual SSH session.

Time estimate: 4–5 hours

12 VMs, configured identically

What You'll Build in This Chapter

Automate post-VM configuration using Ansible. After OpenTofu creates VMs, Ansible installs services, deploys config files, starts services, and hardens systems — without manual SSH.

- All 12 VMs configured identically via Ansible
- Services deployed automatically (BIND9, LLDAP, Authentik, etc.)
- Configuration changes version-controlled
- Entire Phase 1 foundation (Ch. 01–06) deploys in ~30 minutes
- Redeploy or update services without rebuilding VMs

The Analogy

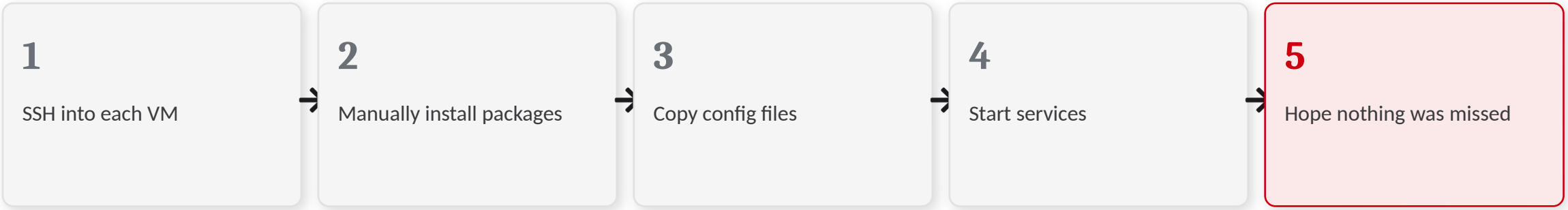
OpenTofu is like hiring a contractor to build 12 empty houses.

Ansible is like hiring a crew to furnish each house identically — furniture, plumbing, electrical — based on one blueprint.

Manual approach isn't repeatable — need 20 VMs instead of 12? Do it all again by hand.

The Problem With Manual Configuration

After Chapter 08 (OpenTofu), we have 12 VMs running — but they're blank.



Problem: Not repeatable. If a VM fails, or you need 20 VMs instead of 12, you're doing all of this again — by hand.

Analogy — *OpenTofu is like hiring a contractor to build 12 empty houses. Ansible is like hiring a crew to furnish each house identically (furniture, plumbing, electrical) based on a blueprint.*

Ansible vs. Other Configuration Management Tools

Tool	Approach	Learning Curve	Agent
Ansible	Declarative, simple	Easy	Agentless (SSH)
Chef	Imperative, powerful	Hard	Agent required
Puppet	Declarative, complex	Hard	Agent required
SaltStack	Imperative, fast	Medium	Agent required



Why Ansible For This Lab

Simple (YAML), agentless (just SSH), and perfect for configuration management at lab scale.



Agentless means: no software to install or maintain on managed nodes. Ansible connects over standard SSH, runs its modules, and leaves — nothing stays resident on the VM.

Install Ansible on Your Host Machine

```
# On your host

$ pip3 install --user ansible


# Verify installation

$ ansible --version
```

→ Expected output: **ansible [core 2.15.x]**

Just Python + pip

Ansible has no server component and no agent to install anywhere. Only your host machine needs the CLI — the 12 managed VMs need nothing but SSH access.

Core 2.15.x is the version used throughout this lab.

Create the Ansible Project Structure

```
$ mkdir -p ~/lab/ansible/devsecops-lab/\

  {inventory,playbooks,roles,templates,files}


$ cd ~/lab/ansible/devsecops-lab
```

```
# Create key files

$ touch ansible.cfg inventory.ini
```

- **playbooks/** Orchestration — site.yml runs all roles
- **roles/** Reusable role definitions per service
- **templates/** Jinja2 templates for config files
- **files/** Static files, scripts & configs

```
devsecops-lab/
├── ansible.cfg
├── inventory.ini
├── playbooks/
│   ├── site.yml
│   ├── dns.yml
│   ├── identity.yml
│   └── edge.yml
├── roles/
│   ├── common/
│   ├── bind9/
│   ├── lldap/
│   ├── authentik/
│   └── haproxy/
├── templates/
│   └── *.j2 config files
└── files/
    ├── scripts/
    └── configs/
```

ansible.cfg — Core Behavior Settings

```
[defaults]
inventory = ./inventory.ini
remote_user = ubuntu
host_key_checking = False
timeout = 30
forks = 10
log_path = ./ansible.log
verbosity = 1

[privilege_escalation]
become = True
become_method = sudo
become_user = root
become_ask_pass = False
```

Inventory & user

Points at inventory.ini and connects as the ubuntu user by default

SSH tuning

Skips host-key prompts, 30s timeout, and 10 parallel forks for speed

Privilege escalation

Auto-becomes root via sudo — no interactive password prompt

Inventory — 12 VMs Across 9 Functional Groups

[dns]
sec-dns-01
10.0.2.10

[identity]
sec-idm-01
10.0.2.11

[edge]
sec-edge-01
10.0.2.12

[secrets]
sec-vault-01
10.0.2.13

[vcs]
sec-vcs-01
10.0.2.14

[appsec]
sec-appsec-01
10.0.2.15

[registry]
sec-registry-01
10.0.2.16

[kubernetes_masters]
k8s-master-01
10.0.2.20

[kubernetes_workers]
k8s-worker-01/02
10.0.2.21–22

[observability]
sec-obs-01
10.0.2.30

[siem]
sec-siem-01
10.0.2.31

[all]
12 hosts total
used by common role

Group-based targeting: each group in inventory.ini lets a playbook target only the hosts that need a specific role — DNS role only touches [dns], edge role only touches [edge].

One Role, Applied to All 12 VMs

Every VM inherits this baseline before any service-specific role runs.

Common packages

curl, git, vim, htop, nmap, ufw, fail2ban, auditd, chrony, jq, python3 + more

Timezone

Sets system timezone to UTC across every host

UFW firewall

Deny-by-default; explicitly allows SSH (port 22)

SSH hardening

No root login, verbose logging, capped auth retries, idle timeout

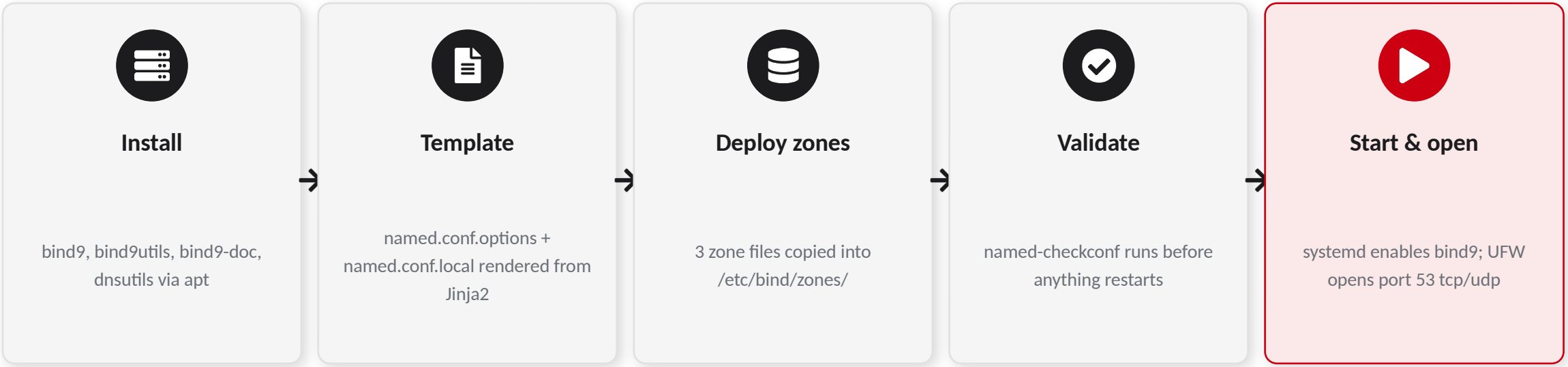
fail2ban

Bans repeat SSH offenders — 5 tries, 1hr ban window

chrony + auditd

Time sync and audit logging enabled and started

Anatomy of a Service Role: BIND9 DNS



Handlers: Notify, Don't Force

Config-deploying tasks use `notify: reload bind9` instead of restarting immediately. The handler only fires once, at the end of the play, and **only if a task actually changed something** — so BIND9 reloads once per run, not once per template file.

site.yml — Orchestrating Roles Across Host Groups

`hosts: all`	→	`roles: common`
`hosts: dns`	→	`roles: bind9`
`hosts: identity`	→	`roles: lldap + authentik`
`hosts: edge`	→	`roles: haproxy + nginx + oauth2-proxy`
`hosts: secrets`	→	`roles: vault`

From Connectivity Test to Full Apply

0. Ping test	`$ ansible all -i inventory.ini -m ping`	*Confirms SSH reaches all 12 VMs*
1. Syntax check	`$ ansible-playbook --syntax-check playbooks/site.yml`	*Catches YAML errors before touching a VM*
2. Dry run	`$ ansible-playbook --check playbooks/site.yml`	*Shows planned changes — nothing is applied*
3. Apply	`$ ansible-playbook playbooks/site.yml`	*Runs for real — takes 10–20 minutes*

Safe to Run Again, Again, and Again

Ansible playbooks are idempotent — running the same playbook twice never causes duplicate or unwanted changes.

FIRST RUN

```
sec-dns-01: ok=10 changed=5

sec-idm-01: ok=15 changed=8

...
```

Packages installed, configs written, services started for the first time. Real work happens.

SECOND RUN

```
sec-dns-01: ok=10 changed=0

sec-idm-01: ok=15 changed=0

...
```

Everything is already in the desired state — Ansible verifies and reports, but changes nothing.

Describe what you want, not how to get there.

Targeted, Flexible Playbook Runs

Run only certain roles

```
$ ansible-playbook -t dns playbooks/site.yml
```

Run only certain hosts

```
$ ansible-playbook -l edge playbooks/site.yml
```

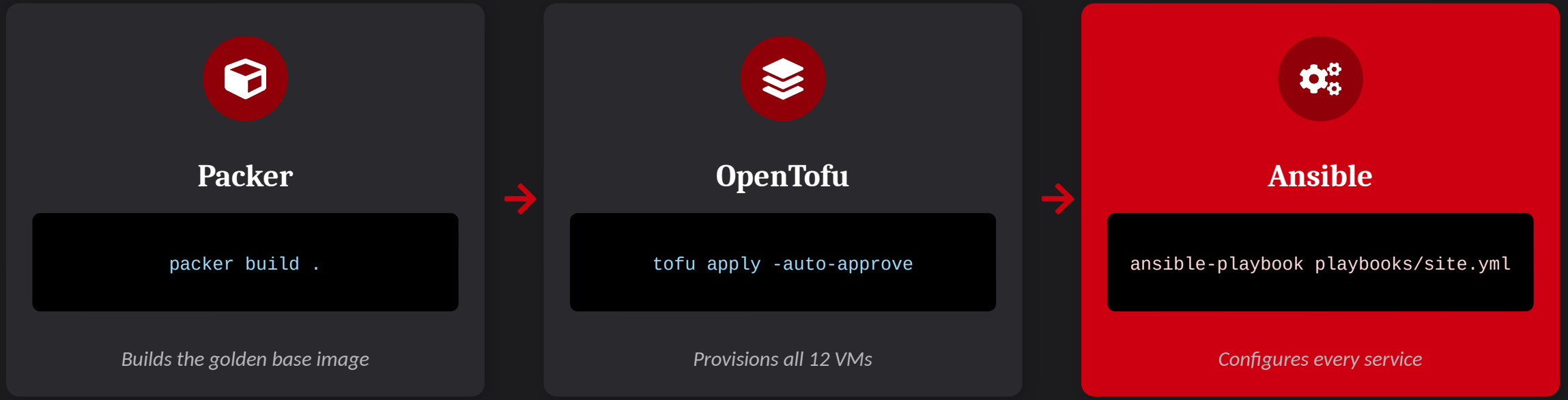
Override variables

```
$ ansible-playbook -e "dns_server=8.8.8.8"
playbooks/site.yml
```

Parallel execution

```
$ ansible-playbook -f 10 playbooks/site.yml
```

Packer → OpenTofu → Ansible



8 hrs → **30 min**

Manual (Ch. 01–06) + human error

Automated + reproducible, every time

Version-controlled · Testable (plan before apply) · Disaster recovery in ~1 hour

Chapter Summary — What We Built

What We Built	Purpose
Ansible project structure	Organized configuration management
Inventory file	Define which VMs to manage
Common role	Base configuration all VMs share
Service roles	Specific configuration per service
Jinja2 templates	Dynamic configuration files
Main playbook	Orchestrate everything
Idempotent design	Safe to run multiple times

Key Takeaway

Phase 2 (Packer → OpenTofu → Ansible) converts manual work into code.

- Version-controlled: track every change in Git
- Reproducible: identical every deployment
- Testable: plan before apply, dry-run first
- Recoverable: disaster recovery in ~1 hour

PHASE 2: COMPLETE

You Now Have a Fully Automated Infrastructure Foundation

- **Packer —** Reproducible golden images
- **OpenTofu —** Infrastructure as code
- **Ansible —** Automated configuration management
- **Full automation —** Lab deploys from scratch in 30 minutes

Where To Next?

1. Phase 3: Vault + Teleport (Secrets & Zero Trust)
2. Phase 4: GitLab + SonarQube + Harbor (CI/CD Supply Chain)
3. Security Series: Deep-dive hardening for Phase 1–2 tools
4. Production Patterns: Multi-region, HA, compliance